LLH Low-Level Security Track: Problem Statement

VaultFS: overview

Build an encrypted filesystem that stays safe when the attacker controls every byte on disk. How you do that is up to you.

Your files sit in a folder synced to a storage provider you do not trust. The provider gets breached. The attacker can now read, copy, edit, delete and restore every stored byte, but they do not have your password. They must learn nothing useful, and they must not be able to change your data without you finding out.

The challenge

Build a system that lets a user unlock a folder on Linux with a password and work in it normally, while everything actually stored stays useless to an attacker. We care about what your system achieves. The design, language and tools are yours to choose.

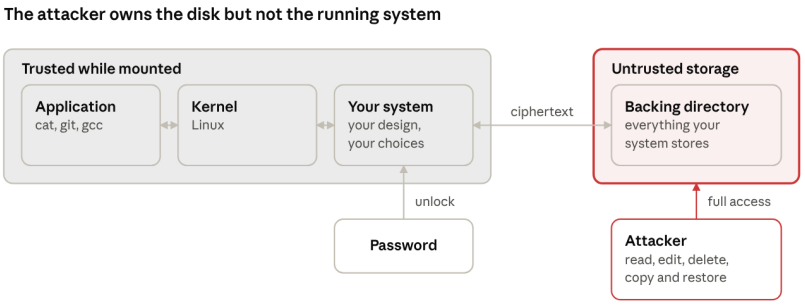
Your system should:

1. **Work like a real folder.** Everyday tools and real workloads run in it without changes.
2. **Give nothing away.** Stored data tells the attacker as little as possible.
3. **Catch tampering.** Altered data never passes as the real thing.
4. **Notice old copies.** Restoring an earlier version of the stored data, even all of it at once, must not go unnoticed.
5. **Protect the password.** A wrong password gets a clear refusal, stolen data is hard to crack, and users can change their password.
6. **Support multiple users.** Each user has their own password and owns their files. Users can read only their own files and what is shared with them, and this holds even if they read the stored data directly. Users can share files and folders and remove that access. Removing access locks that person out of everything written afterwards, without re-encrypting the whole volume.
7. **Keep secrets safe while running.** Passwords and keys stay protected even while the program is running.
8. **Survive failure.** A crash does not cost the user their data.
9. **Hold up under attack.** Your own code stays safe when it reads data an attacker wrote.

How far you take each goal, and how you trade them off, is your call.

The attacker

The attacker has full control of your stored data and can keep old copies of it. They do not have your password or access to your running system.



Everything beyond that is yours to define.

Go further

The best entries will surprise us. Think about what an attacker would try next and what a real user would need, then build what you think matters most.

Submission

Submit one GitHub repository. Everything must be inside it.

Item	What your repository must include
Code	All source code, with its commit history
Run instructions	All dependencies and setup steps, clearly listed, plus how to build it, how to create, mount and unmount a volume change its password, add and remove users, share files, and which Linux distributions it works on
Evidence	The tests and benchmarks that show it works
Demo	A link to a 5-minute video of the working system

Evaluation

Total score: 100.

Criterion	Score
Correctness	20
Robustness	20
Technical depth	20
Innovation	15
Performance	10
Demonstration	10
Code quality	5

Rules

Any language, public library, tool or documentation is allowed, including libfuse3 or other libraries for implementing your filesystem. The filesystem itself must be your own work, not a fork or wrapper of an existing encrypted filesystem.

You may be required to give a short viva if the organizers request one. Failing to do so may result in your submission being rejected.